

자료구조응용 과제

과제 제출 시 주의 사항:

1. 제출하는 프로그램은 Visual studio 2026 에서 수행되어야 한다.
2. 한 개의 c (또는 cpp) 프로그램으로 작성하며, header file 없이 한 개의 c (또는 cpp) file 만으로 구성되게 한다. 프로그램 file 은 <ds 과제물번호_이름.c> (또는 .cpp)의 이름을 가져야 한다.
예) 홍길동이 과제 #02 을 제출할 경우, 'ds02_홍길동.c' 또는 'ds02_홍길동.cpp'가 되어야 함.

3. 학과, 학번, 이름이 프로그램의 실행 화면 첫줄에 출력되게 한다.

**** 과제는 해당 수업 시간 내에 코딩하여 튜터에게 검사/채점 받고 lms 에 제출해야 배점이 됨.**

- 튜터에게 검사/채점 받지 않고 수업시간 내에 lms 에 제출한 경우 0 점 처리함.
- 수업 시간 내에 코딩이 완료되지 않은 경우 1) 미완료된 부분, 이유, 오류 발생 부분 등에 대해 프로그램 내에 COMMENT 문으로 작성한 뒤 2) 이것을 수업시간 내에 튜터에게 검사 받고 lms 에 제출해야 차후 코딩을 완료하여 늦은 제출(기한:수업한 날의 밤 11:59)을 할 수 있음.
- 튜터에게 검사 받지 않고 중도 퇴실한 경우에는 결석으로 처리하고 과제도 0 점 처리됨.
- AI 가 생성한 코드로 의심되는 경우 0 점 처리함.

채점방법

1. 위의 지시사항을 따르지 않으면 0 점 처리합니다.
2. 입출력 양식을 지키지 않으면 0 점 처리합니다.

주의!!!! * 응용 수업 시간을 마칠 때, 본인이 작성 (및 제출)한 과제 프로그램이 실습실의 PC 에 남아 있지 않도록 삭제해야 함. 이를 어겨서 copy 등의 문제가 생길 경우 0 점 처리함.

과제 #3 (만점: 10 점)

단계 1) 첨부된 파일 in.txt 에는 오름차순의 중복 없는 정수들이 저장되어 있다. in.txt 에 저장된 정수의 개수 (L_size)를 알아낸 뒤, L_size 개의 정수들을 저장할 수 있는 array L 을 malloc 으로 생성하라. 그후, 파일 in.txt 로부터 정수를 차례대로 읽어 들여 array L 에 저장한다.

- feof 를 사용할 것.
- File 에 저장된 모든 정수의 개수(L_size)를 알아내기 위해서는 file 을 open 하여 모든 정수를 차례대로 읽으며 count 해야 하고, 이때 file pointer 는 file 의 끝까지 가게 됨. File pointer 를 file 의 시작 위치로 다시 이동하기 위해서는 두 가지 방법이 가능함.
 - L_size 를 알아내기 위해 file open/close 를 한번 실행하고, 그 뒤 in.txt 의 정수들을 읽어들이기 위해 file open/close 를 한번 더 실행하는 방법이 있음.
 - fseek 을 이용하여 file open 상태에서 pointer 를 초기 위치로 이동할 수도 있음.

단계 2) 위에서 생성한 array L 을 이용하여 sequential search, iterative binary search, recursive binary search 의 time complexity 를 비교하고자 한다. 0 부터 freq-1 까지의 모든 정수를 차례대로 array L 에서 search 하며, search 한 결과를 모두 출력하면 출력량이 너무 많기 때문에 0 부터 4 까지 search 한 결과만 출력하여 실행이 바르게 되었는지 확인한다.

function seqsearch(), iter_binsearch(), rec_binsearch()가 각각 sequential search, iterative binary search, recursive binary search 를 수행하도록 작성하고 아래의 코드를 완성하라.

```
int main()
{
    // 학부, 학번, 이름 출력

    // 단계 1 작성 부분

    // 단계 2

    int freq = 100000;

    printf("sequential search:\n");
```

```

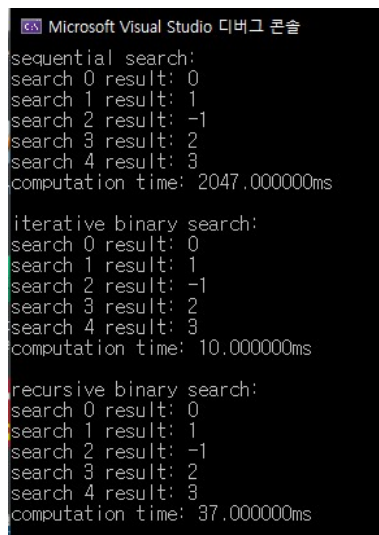
start = clock();
for (int i = 0; i < freq; i++) {
    int res = seqsearch(L, i, L_size);
    if (i < 5) printf("search %d result: %d\n", i, res);
}
end = clock();
printf("computation time: %fms\n", 1000 * ((double)end - (double)start) / CLOCKS_PER_SEC);

printf("iterative binary search:\n");
// 위의 코드를 참고하여 작성

printf("recursive binary search:\n");
// 위의 코드를 참고하여 작성

return 0;
}
실행 예제

```



```

Microsoft Visual Studio 디버그 콘솔
sequential search:
search 0 result: 0
search 1 result: 1
search 2 result: -1
search 3 result: 2
search 4 result: 3
computation time: 2047.000000ms

iterative binary search:
search 0 result: 0
search 1 result: 1
search 2 result: -1
search 3 result: 2
search 4 result: 3
computation time: 10.000000ms

recursive binary search:
search 0 result: 0
search 1 result: 1
search 2 result: -1
search 3 result: 2
search 4 result: 3
computation time: 37.000000ms

```